

Delta Table Migration Orchestrator

Design Document — Cross-Region Delta Table Deep Clone Framework

Purpose: A notebook- and job-based orchestration framework for migrating Delta tables between Databricks workspaces in different Azure regions using either Delta Sharing or direct ADLS paths.

Document	Delta Table Migration Orchestrator — Technical Design
Version	1.0
Primary execution model	Databricks Notebook + Databricks Job
Clone engines	Delta Sharing Deep Clone; Direct ADLS Deep Clone
Operating modes	Inventory / Onboarding; Deep Clone; Validate; Retry
Input mechanisms	Job parameters; YAML; CSV

Design principle: Treat this as a migration orchestration framework, not merely a Deep Clone script. Inventory, state management, workload scheduling, retries, validation and audit form the control plane; Deep Clone is the execution engine.

1. Objectives and Scope

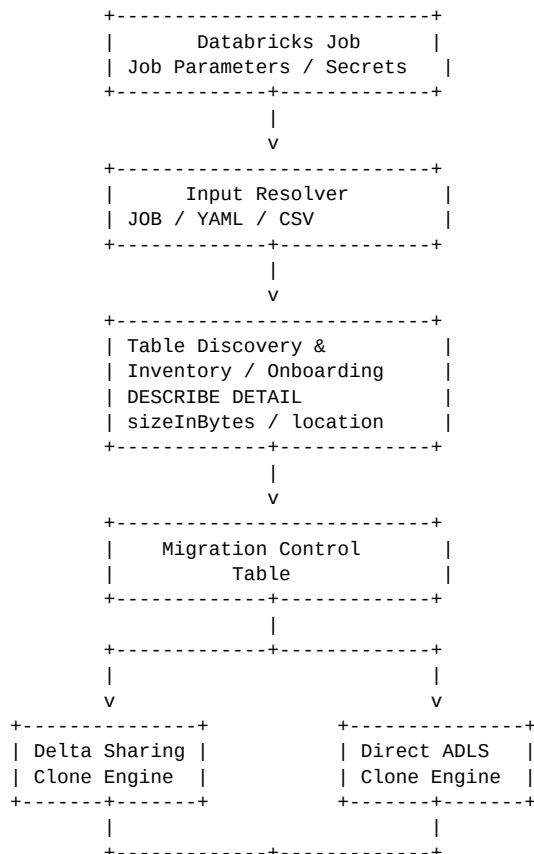
The framework provides controlled, restartable and auditable migration of Delta tables from a source Databricks workspace to a target workspace located in another region.

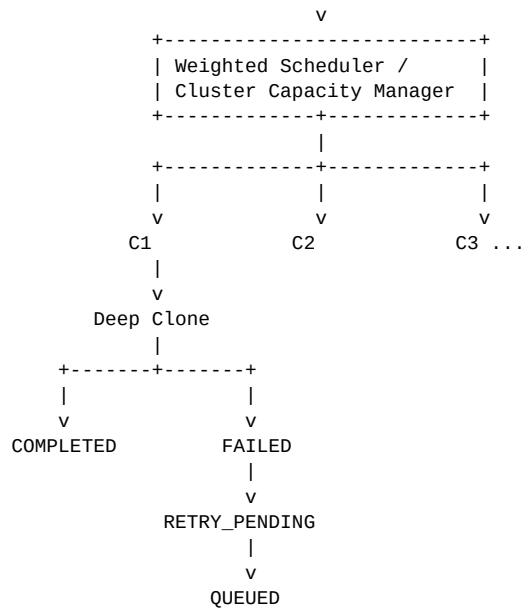
- Support two clone flavours: Deep Clone through Delta Sharing and Deep Clone directly from source ADLS paths.
- Support catalog-, schema-, and table-level selection.
- Support selection through job parameters, YAML configuration, or CSV mapping.
- Allow explicit job parameters to override configurable runtime settings.
- Inventory source tables before execution, including sizeInBytes and source ADLS location.
- Classify tables by workload size and schedule them across a pool of existing all-purpose compute clusters.
- Maintain exact per-table state, execution attempts, errors, assigned cluster, and validation status.
- Support safe retry without rediscovering or reprocessing the complete migration scope.
- Provide a validation phase independent from cloning.

2. Non-Goals

- The framework does not provision the Databricks workspaces, clusters, VNets, private endpoints, storage accounts, or Unity Catalog infrastructure.
- The source SQL Warehouse is used for metadata discovery in Direct ADLS mode; it is not the Deep Clone execution engine.
- The scheduler does not attempt to dynamically resize clusters. It schedules work against a configured logical cluster capacity.
- Row-count validation is optional because COUNT(*) over very large tables can create significant additional workload.

3. High-Level Architecture





4. Operating Modes

Mode	Purpose	Writes / Changes
INVENTORY	Resolve selection, discover tables, capture metadata, classify workload, populate inventory records.	Inventory records only
DEEP_CLONE	Consume queued records and execute clone operations through the selected target.	Creates target Delta tables
VALIDATE	Compare source and target metadata and optionally row counts.	Validation status
RETRY	Requeue eligible failed or validation-failed records without rediscovery.	Attempt/state updates

5. Clone Flavours

5.1 Delta Sharing Deep Clone

The source table is exposed through Delta Sharing and the target workspace executes Deep Clone against the shared source table identifier.

```
CREATE TABLE <target_catalog>.<target_schema>.<target_table>
DEEP CLONE <source_catalog>.<source_schema>.<source_table>;
```

The inventory process resolves the requested catalog/schema/table scope and creates one migration-control record per table.

5.2 Direct ADLS Deep Clone

For direct ADLS mode, the source table location is obtained during inventory by executing DESCRIBE DETAIL against the source SQL Warehouse. The warehouse ID is therefore a required configuration parameter for this clone flavour.

```
DESCRIBE DETAIL ril_curated.gold.customer_360
```

At minimum, inventory captures **location** and **sizeInBytes**. The actual Deep Clone is executed from the target-side worker compute using the recorded source path.

```
CREATE TABLE <target_catalog>.<target_schema>.<target_table>
DEEP CLONE '<source_adls_path>';
```

The source warehouse is deliberately separated from the execution path: it performs metadata discovery only.

6. Input and Selection Model

6.1 Selection Granularity

Selection	Behaviour
CATALOG	Discover all schemas and eligible tables under each source catalog and map to the configured target catalog.
SCHEMA	Discover all eligible tables under the specified source schema and map to the target schema.
TABLE	Clone only explicitly listed source-to-target table mappings.

6.2 Input Mechanisms

Input	Recommended use
JOB	Small or dynamically supplied migrations; highest runtime precedence.
YAML	Reusable migration plans, catalog/schema mappings, execution settings.
CSV	Explicit table-by-table source-to-target mapping supplied by migration teams.

6.3 CSV Format

```

source_catalog,source_schema,source_table,target_catalog,target_schema,target_table
ril_curated,gold,customer_360,ril_curated,gold,customer_360
ril_curated,gold,vehicle,ril_curated,gold,vehicle
ril_bronze,raw,events,ril_bronze,raw,events

```

6.4 YAML Example

```

migration:
  clone_type: direct_adls
  source_warehouse_id: abc123

selection:
  type: catalog
  catalogs:
    - source: ril_curated
      target: ril_curated
    - source: ril_bronze
      target: ril_bronze

execution:
  max_retries: 3
  validation_enabled: true

cluster_pool:
  - cluster_id: cluster-1
    capacity_units: 20
  - cluster_id: cluster-2
    capacity_units: 10
  - cluster_id: cluster-3
    capacity_units: 5

```

6.5 Parameter Precedence

To make execution deterministic, use the following precedence:

```

Explicit Job Parameters
    |
    v
YAML configuration
    |
    v
CSV table mapping
    |
    v
Framework defaults

```

The selected input source should be explicit through an `input_type` parameter. Runtime infrastructure settings such as cluster pool, retry count and source warehouse ID may be overridden by explicit job parameters.

7. Inventory / Onboarding Design

Inventory is the control-plane entry point. It must be idempotent and must not execute a clone.

1. Resolve requested catalogs / schemas / tables.
2. Validate source object existence.
3. For each table, execute DESCRIBE DETAIL where required.
4. Capture source location and sizeInBytes.
5. Capture source metadata needed for validation.
6. Resolve target mapping.
7. Classify workload.
8. Upsert the migration-control record.
9. Mark eligible records WAITING_FOR_LOAD.

7.1 Inventory Metadata

Metadata	Purpose
source_catalog/schema/table	Source object identity
target_catalog/schema/table	Destination identity
source_path	Required by Direct ADLS clone
size_in_bytes	Workload classification and scheduling
num_files	Validation and operational metrics
format	Confirm Delta source
created_at / last_modified	Audit and source-state tracking
source_version	Optional validation/reference metadata

8. Workload Classification and Load Balancing

Scheduling should be based on workload units rather than a fixed number of threads. This avoids treating a 5 GB table and a 5 TB table as equivalent work.

Class	Example size	Default weight
SMALL	< 10 GB	1
MEDIUM	10–100 GB	3
LARGE	100 GB–1 TB	8
XLARGE	> 1 TB	20

Thresholds and weights should be configurable. The values above are starting defaults, not hard-coded assumptions.

8.1 Cluster Pool

The framework maintains a logical pool of pre-existing all-purpose clusters. Each cluster is assigned a capacity in workload units.

Cluster	Capacity	Example
Cluster-1	20 units	1 XLARGE or 2 LARGE + small work
Cluster-2	10 units	1 LARGE + medium/small work
Cluster-3	5 units	Medium/small tables

8.2 Scheduling Algorithm

1. Select QUEUED / RETRY_PENDING tables.
2. Order by workload_weight DESC.

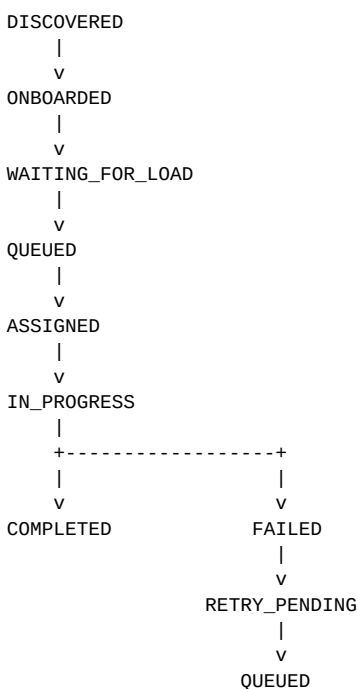
3. Inspect available capacity for every cluster.
4. Select the least-utilized cluster that can fit the table.
5. Atomically mark the table ASSIGNED and record cluster_id.
6. Worker changes ASSIGNED -> IN_PROGRESS.
7. On completion, release capacity and mark COMPLETED.
8. On failure, release capacity and mark FAILED / RETRY_PENDING.

The scheduler should be work-conserving: when capacity becomes available, the next fitting table should be dispatched rather than waiting for a fixed batch boundary.

8.3 Concurrency Model

Use Python threads or asynchronous orchestration only for control-plane coordination. Do not build the framework around a single driver executing dozens of heavy clone operations through a ThreadPoolExecutor. Each actual table clone should execute as an isolated worker on a selected compute cluster. This gives cleaner resource isolation, failure handling and observability.

9. State Machine



Validation may transition:
COMPLETED -> VALIDATION_FAILED -> RETRY_PENDING

Recommended terminal states are COMPLETED, FAILED_PERMANENT and SKIPPED. A validation failure should be distinguished from an execution failure.

10. Data Model

10.1 Migration Control Table

```

CREATE TABLE migration_control (
  migration_id STRING,
  run_id STRING,
  clone_type STRING,

  source_workspace STRING,
  target_workspace STRING,

  source_catalog STRING,
  source_schema STRING,
  source_table STRING,

  target_catalog STRING,
  target_schema STRING,
  target_table STRING,

```

```

source_path STRING,

size_in_bytes BIGINT,
size_gb DOUBLE,
workload_class STRING,
workload_weight INT,

assigned_cluster_id STRING,
status STRING,

attempt_number INT,
max_attempts INT,

discovered_at TIMESTAMP,
onboarded_at TIMESTAMP,
queued_at TIMESTAMP,
started_at TIMESTAMP,
completed_at TIMESTAMP,
failed_at TIMESTAMP,

duration_seconds BIGINT,

error_code STRING,
error_message STRING,

source_num_files BIGINT,
target_num_files BIGINT,
source_version BIGINT,
target_version BIGINT,

validation_status STRING,
validation_message STRING,

created_at TIMESTAMP,
updated_at TIMESTAMP
)

```

10.2 Migration Attempts Table

```

CREATE TABLE migration_attempts (
  run_id STRING,
  migration_id STRING,
  attempt_number INT,

  cluster_id STRING,
  worker_id STRING,

  started_at TIMESTAMP,
  completed_at TIMESTAMP,

  status STRING,

  error_code STRING,
  error_message STRING,

  duration_seconds BIGINT,

  source_size_bytes BIGINT,
  target_size_bytes BIGINT,

  created_at TIMESTAMP
)

```

The control table stores the current state of each table. The attempts table preserves execution history and prevents later retries from overwriting the evidence of earlier failures.

11. Deep Clone Execution

11.1 Worker Contract

A worker receives a migration-control record and is responsible for one table execution.

```
Input:
  migration_id
  clone_type
  source identifier/path
  target catalog/schema/table
  assigned cluster
  attempt number

Actions:
  1. Validate input state.
  2. Atomically mark IN_PROGRESS.
  3. Execute Deep Clone.
  4. Capture execution metrics.
  5. Update migration_attempts.
  6. Update migration_control.
  7. Release scheduler capacity.
```

11.2 Idempotency

- Do not blindly issue CREATE TABLE for an existing target.
- Before execution, determine whether the target exists and whether its state is consistent with the migration-control record.
- A completed migration should not be re-executed by a normal DEEP_CLONE run.
- Explicit RETRY should handle failed or validation-failed records.
- Use atomic state transitions so two scheduler instances cannot acquire the same table.

12. Validation Design

Validation is an independent mode so that cloning and verification can be scaled separately.

Validation	Default	Purpose
Source exists	Yes	Confirm source object availability
Target exists	Yes	Confirm target creation
Size	Yes	Compare source/target physical size where meaningful
File count	Yes	Detect obvious incomplete copies
Delta metadata/version	Recommended	Confirm expected Delta state
Row count	Optional	Stronger logical validation but potentially expensive

Validation should record both validation_status and validation_message. A failed validation should not be silently converted into COMPLETED.

13. Retry Design

Retry operates only on the existing control records. It must not repeat the inventory/discovery process unless explicitly requested.

```
Eligible records:
  status IN ('FAILED', 'RETRY_PENDING', 'VALIDATION_FAILED')
  AND attempt_number < max_attempts
```

```
Transition:
  FAILED / VALIDATION_FAILED
    -> RETRY_PENDING
    -> QUEUED
```

- > ASSIGNED
- > IN_PROGRESS
- > COMPLETED or FAILED

- Increment attempt_number before dispatch.
- Preserve previous error details in migration_attempts.
- Support a configurable maximum retry count.
- Optionally implement exponential backoff for transient failures.
- Permanent errors should transition to FAILED_PERMANENT after the retry limit or explicit classification.

14. Job / Notebook Design

The implementation should remain notebook-based, with the notebook acting as the orchestration entry point. Supporting Python modules can be packaged alongside the notebook in the workspace/repository.

Module	Responsibility
config	Runtime configuration and defaults
parameter_parser	Databricks job parameters and precedence
input_resolver	JOB/YAML/CSV processing
table_discovery	Catalog/schema/table enumeration
inventory_manager	DESCRIBE DETAIL and control-table onboarding
workload_classifier	Size-to-weight classification
scheduler	Queue selection and cluster assignment
cluster_pool	Capacity accounting
clone_worker	Common worker contract
shared_clone_worker	Delta Sharing Deep Clone
adls_clone_worker	Direct ADLS Deep Clone
validator	Post-clone validation
retry_manager	Retry eligibility and requeue
audit_manager	State and attempt history

15. Suggested Job Parameters

```

mode
clone_type
input_type
selection_type

source_workspace_url
target_workspace_url
source_warehouse_id

input_path
source_catalogs
source_schemas
source_tables

cluster_pool_config
max_retries
validation_enabled
row_count_validation

run_id

```

The exact Databricks parameter encoding can be JSON strings, comma-separated lists, or paths to configuration files. JSON is preferred for complex mappings because catalog-to-target and table-to-target relationships become unambiguous.

16. Operational Metrics

At minimum, expose the following metrics from the control and attempts tables:

- Total tables discovered / onboarded / queued / in progress / completed / failed.
- Total source bytes and bytes completed.
- Tables by workload class.
- Current cluster utilization in workload units.
- Average and percentile clone duration by workload class.
- Failure count by error code.
- Retry count and success-after-retry rate.
- Validation success/failure count.
- Throughput: bytes migrated per hour.

17. Failure Scenarios and Handling

Scenario	Expected behaviour
Source table not found	Inventory record marked FAILED_PERMANENT / discovery failure; no clone dispatched.
Source path unavailable	Clone attempt fails; capture error; retry if classified transient.
Target already exists	Worker checks target state and avoids accidental overwrite.
Cluster unavailable	Scheduler removes/unavailable-flags capacity and requeues assigned work.
Clone timeout/failure	Attempt recorded; table moves to RETRY_PENDING or FAILED.
Validation failure	Target remains visible but migration status is VALIDATION_FAILED; eligible for retry/review.
Scheduler restart	Control table is source of truth; stale IN_PROGRESS/ASSIGNED records can be reconciled.

18. Security and Access Considerations

- Use Databricks Secrets for OAuth credentials or other credentials where required; never persist secrets in YAML or control tables.
- The source warehouse identity must have permission to DESCRIBE the source tables in Direct ADLS mode.
- The target execution identity must have the required access to the target catalog/schema and source storage path for Direct ADLS mode.
- Delta Sharing mode requires the target workspace/principal to have access to the shared source tables.
- Control and attempts tables should be protected because they contain workspace, storage-path and operational metadata.
- Do not place access tokens, SAS tokens or client secrets in audit records.

19. Recommended Execution Lifecycle

```
PHASE 1 – INVENTORY
  Resolve selection
    -> Discover tables
    -> DESCRIBE DETAIL
    -> Capture path + size
```

```
-> Classify workload
-> ONBOARDED / WAITING_FOR_LOAD
```

PHASE 2 – DEEP CLONE

```
Select QUEUED records
-> Weighted scheduler
-> Assign cluster capacity
-> Worker executes clone
-> COMPLETED / FAILED
```

PHASE 3 – VALIDATE

```
Select completed records
-> Source/target checks
-> VALIDATED / VALIDATION_FAILED
```

PHASE 4 – RETRY

```
Select eligible failures
-> Increment attempt
-> QUEUE
-> Deep Clone
-> Validate
```

20. Design Decisions

Decision	Rationale
Control table is source of truth	Makes the framework restartable and operationally observable.
Separate attempts table	Preserves complete failure/retry history.
Inventory before clone	Decouples discovery from execution and makes scheduling possible.
Size-based workload units	Prevents small and very large tables from receiving equal scheduling weight.
Logical cluster pool	Allows heterogeneous existing clusters to participate without provisioning logic.
Source warehouse only for metadata	Avoids coupling metadata discovery with the heavy clone execution path.
Worker per table	Improves isolation, retryability and auditability.
Explicit input_type	Prevents ambiguous merging of YAML, CSV and job parameters.

21. Future Extensions

- Add AzCopy as a third execution engine while retaining the same control plane.
- Add estimated-duration models based on historical clone performance rather than only table size.
- Add dependency-aware scheduling for tables that must migrate in a specific order.
- Add pause/resume controls and cluster maintenance windows.
- Add migration waves or business-unit grouping.
- Add dashboarding over migration_control and migration_attempts.
- Add pre-flight checks for storage connectivity, permissions, UC objects and target readiness.

22. Final Architecture Recommendation

The framework should be implemented as a **Delta Table Migration Orchestrator** with a stable control plane and pluggable clone engines. The control plane owns selection, inventory, state, workload classification, scheduling, retries, validation and audit. The execution layer owns the mechanics of cloning.

This architecture allows the current two flavours — Delta Sharing Deep Clone and Direct ADLS Deep Clone — to share nearly all orchestration logic. The result is a reusable migration framework rather than two independent cloning utilities.